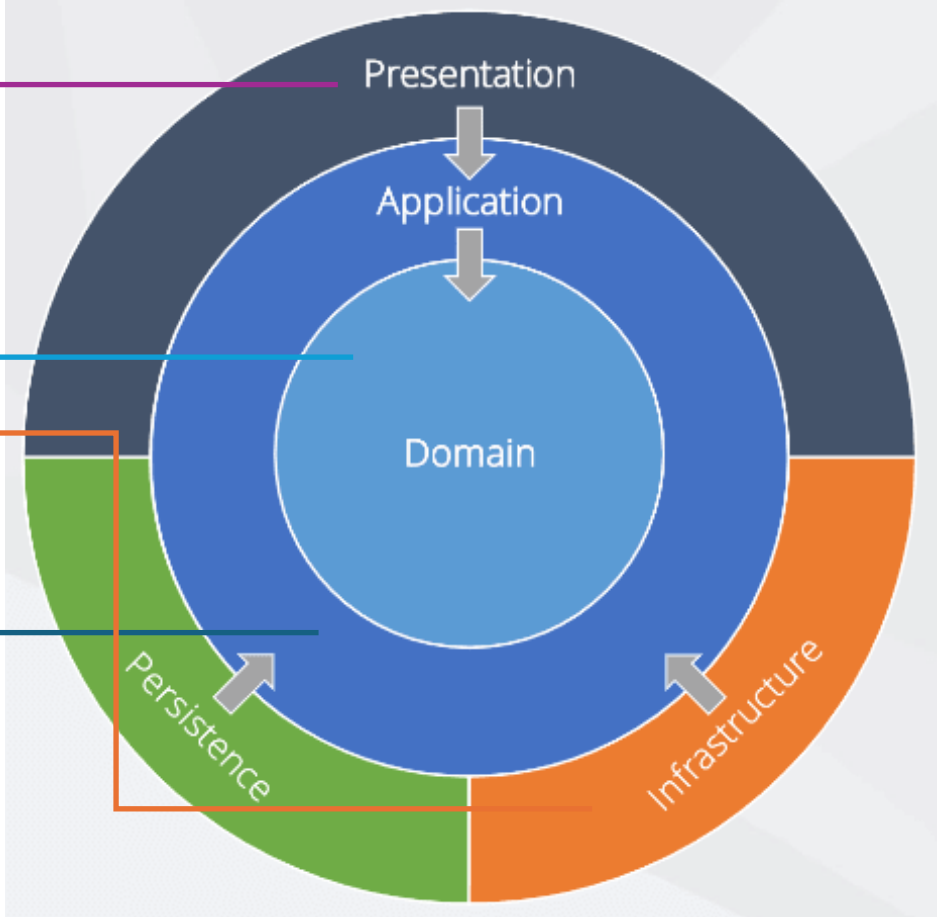
API Eventhub

Présentation de la semaine et développement du backend

Architecture du projet

```
app.ts

api
├── controllers
│   │   event.controller.ts
│
├── middlewares
│   │   authentication.middleware.ts
│   │   error-handler.middleware.ts
│   │   index.ts
│   │   json-api-response.middleware.t
│
├── prisma
│   │   client.ts
│   │   seed.ts
│
├── routes
│   │   eventRoute.ts
│   │   index.ts
│
└── utility
    │   index.ts
    │   password.utility.ts
    │   sanitizer.ts
    │   utils.ts

application
└── usecases
    ├── events
    │   │   CreateEventUseCase.ts
    │   │   DeleteEventUseCase.ts
    │   │   GetAllEventUseCase.ts
    │   │   GetEventByIdUseCase.ts
    │   │   index.ts
    │   │   UpdateEventUseCase.ts
    │
    └── users
        │   GetProfileUseCase.ts
        │   index.ts
```

```
docs
│   swagger.ts
│
├── paths
│   │   events.delete.ts
│   │   events.get.ts
│   │   events.post.ts
│   │   events.put.ts
│
└── schemas
    │   createEventDTO.schema.ts
    │   event.schema.ts
    │   UpdateEventDTO.schema.ts

domain
├── entities
│   │   Event.ts
│   │   User.ts
│
└── interfaces
    │   EventRepositoryInterface.ts
    │   UserRepositoryInterface.ts

infrastructure
└── repositories
    │   eventRepositoryDatabase.ts
    │   InMemoryEventRepository.ts

tests
└── unit
    │   CreateEventUseCase.test.ts
```



Design Patterns utilisés

- **Repository Pattern**
  - Interface `EventRepositoryInterface` + implémentation `EventRepositoryDatabase`
  - Avantage : Découple logique métier.
- **Dependency Injection (DI)**
  - Exemple : Injection du repository dans le Use Case (`new CreateEventUseCase(repository)`)
  - Avantage : Testabilité.
- **DTO (Data Transfer Object)**
  - Exemple : `CreateEventDTO`
  - Avantage : Standardise les données échangées entre couche API et Use Case.

SOLID - Single Responsibility Principle (SRP)

- Chaque **Use Case** a une seule responsabilité (CreateEventUseCase, UpdateEventUseCase...).
- Chaque **Controller** gère uniquement les requêtes HTTP, pas la logique métier.
- Chaque **Repository** gère uniquement l'accès aux données.

SOLID - Open/Closed Principle (OCP)

- On peut créer un nouveau repository (ici : `InMemoryEventRepository` pour les tests) **sans modifier les use cases existants**.

SOLID - Liskov Substitution Principle (LSP)

- Tous les **repositories** implémentent `EventRepositoryInterface`.
- On peut remplacer `EventRepositoryDatabase` par `InMemoryEventRepository` dans les tests **sans changer les use cases**.

SOLID - Interface Segregation Principle (ISP)

- `EventRepositoryInterface` définit seulement les méthodes nécessaires (`findById`, `create`, `update`, `delete`).
- Les use cases ne dépendent que de ce dont ils ont besoin.

SOLID - Dependency Inversion Principle (DIP)

- Use cases et controllers dépendent de **interfaces**, pas des classes concrètes.
- Les classes concrètes (`EventRepositoryDatabase`) sont injectées via DI.

API REST
(REpresentational State Transfer)

- **Définition :** Interface pour communiquer avec le backend via HTTP.
- **Endpoints principaux :**
  - `POST /api/events/create` – Créer un événement
  - `GET /api/events` – Lister tous les événements
  - `GET /api/events/:id` – Récupérer un événement
  - `PUT /api/events/:id` – Mettre à jour un événement
  - `DELETE /api/events/:id` – Supprimer un événement

```
model User {
  id        String @id @default(uuid())
  firstName String
  lastName  String
  email     String @unique
  phone     String
  role      String
  password  String
  salt      String
}

model Event {
  id          String      @id @default(uuid())
  title       String
  description String
  date        DateTime
  capacity    Float
  price       Float
  imageUrl    String[]    @default([])
  organizer   String
  venue       String
  category    String
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
}
```

Accès aux données / BDD

```
const events = [
    {
        "title": "Titre de l'event",
        "description": "Event de test",
        "date": new Date(Date.now() + 86400000),
        "venue": "10 Rue de la Roquette",
        "category": "Api",
        "capacity": 100,
        "organizer": "Damien J",
        "price": 50
    },
    {
        "title": "Concert Rock",
        "description": "Un super concert",
        "date": new Date(Date.now() + 86400000), // demain
        "venue": "Salle de Concert",
        "category": "Musique",
        "capacity": 200,
        "organizer": "Damien J",
        "price": 99,
        "imageUrl": [],
    },
    {
        "title": "Concert Jazz",
        "description": "Un super concert",
        "date": new Date(Date.now() + 2 * 86400000), // dans 2 jours
        "venue": "Salle de Concert",
        "category": "Musique",
        "capacity": 200,
        "organizer": "Damien J",
        "price": 75,
        "imageUrl": [],
    }
]
```

Accès aux données / BDD

```
const users = [
    {
        "firstName": "Damien",
        "lastName": "J",
        "phone": "0123456789",
        "email": "damien@gmail.com",
        "role": "admin",
        "password": "azerty",
    },
    {
        "firstName": "Lea",
        "lastName": "M",
        "phone": "0123456789",
        "email": "lea@gmail.com",
        "role": "user",
        "password": "qwerty",
    }
]
```

Accès aux données / BDD

```
PS D:\3WA\2025\Projet_Final\event-hub\eventhub_backen

> eventhub_backend@1.0.0 seed
> tsx ./src/api/prisma/seed.ts

Utilisateur damien@gmail.com créé !
Utilisateur lea@gmail.com créé !
Event Titre de l'event créé !
Event Concert Rock créé !
Event Concert Jazz créé !
Tout est créé !
```

Démonstration feature “Gestion des événements”

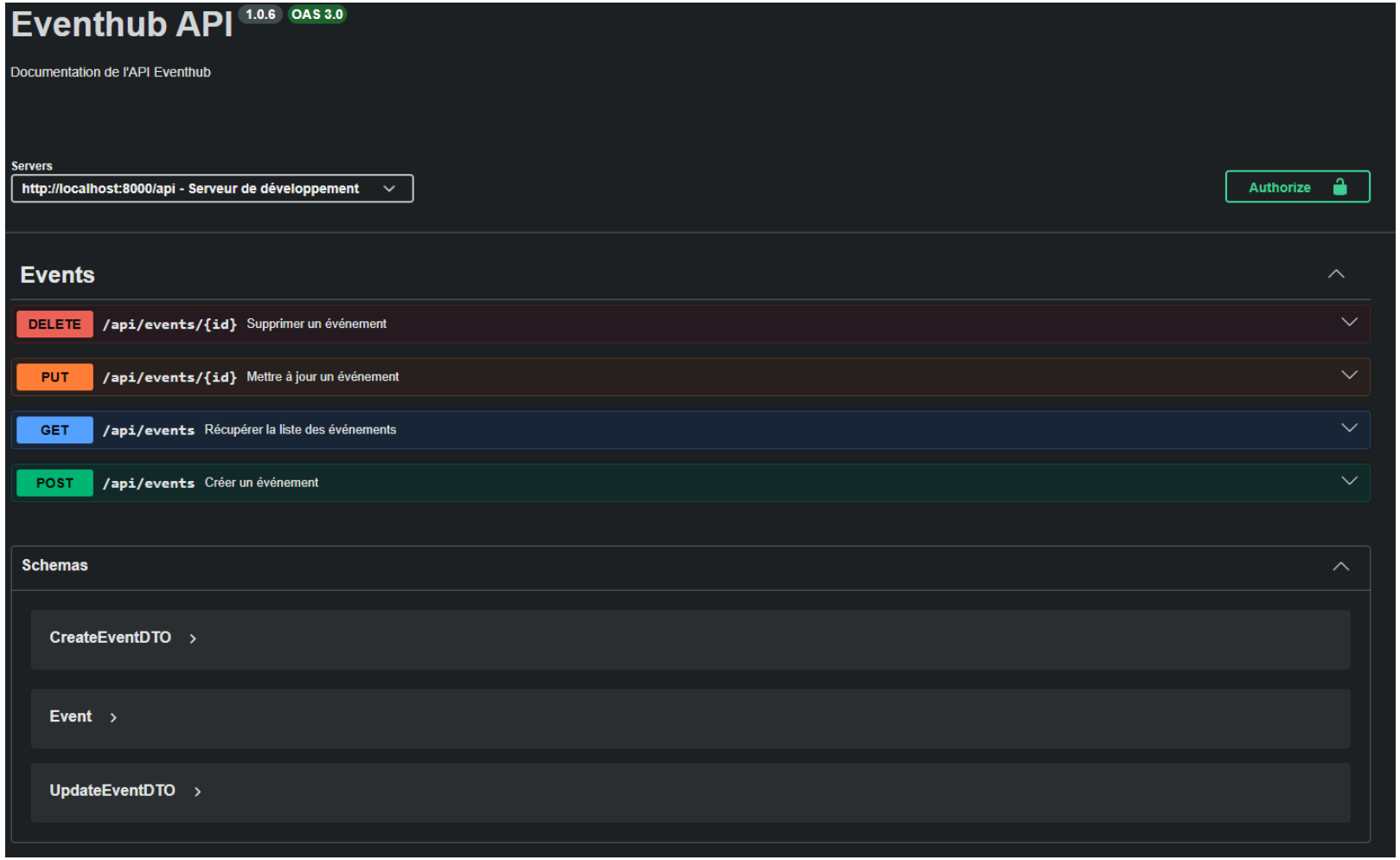
Tests avec Jest

- Approche TDD - Test-Driven Development

```
PASS  src/tests/unit/CreateEventUseCase.test.ts
  CreateEventUseCase
    √ should create an event successfully (2 ms)
    √ should throw an error if title is missing (4 ms)
    √ should throw an error if description is missing (1 ms)
    √ should throw an error if venue is missing (1 ms)
    √ should throw an error if category is missing (1 ms)
    √ should throw an error if price is missing (1 ms)
    √ should throw an error if capacity is under 0 (1 ms)
    √ should throw an error if date is in the past (3 ms)

Test Suites: 1 passed, 1 total
Tests:       8 passed, 8 total
Snapshots:   0 total
Time:        1.239 s, estimated 2 s
Ran all test suites.
```

Documentation technique

htpp://localhost:8000/api-docs



Conclusion

- **Compétences acquises :**
  - Architecture en couches
  - Design patterns (Repository, DI, DTO)
  - API REST sécurisée
  - Express approfondi
  - Prisma et PosgreSQL
- **Difficultés rencontrées :**
  - Gestion des types TypeScript avec Prisma
  - Relations BDD
  - Import/liens entre les différentes couches
- **Points d'amélioration :**
  - Gestion des utilisateurs/roles plus avancée
  - Pagination et filtres sur les events

Merci !

https://github.com/xdam99/event-hub/tree/dev/eventhub_backend